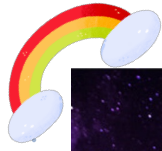


Prime Numbers



"At the end of the day, let
there be no excuses, no
explanations, no regrets."

— Steve Maraboli



Today's content

} Doubt session on
Saturday → 11 AM }

- Prime number Intro
- Get all primes from 1 to N
- Print smallest prime factor for 2 to N
- Prime factorisation
- Get the no. of factors/divisors

Prime no. $\rightarrow$ No. having only 2 factors
1 & itself

Eg: 2, 3, 5, 7, 11, 13, 17, 19 ..

Q Given a no., check if it is prime or not.

Ans = Count the no. of factors

if (count > 2) $\rightarrow$ not prime

else if (count == 2) $\rightarrow$ prime.

boolean checkprime (int N)

count = 0

for (i = 1 ; i * i $\leq$ N ; i++) {

if (N % i == 0) {

if (i == N/i) count ++;

else count += 2

}

}

if (count == 2) return true;

else return false

TC: $O(\sqrt{n})$

SC: $O(1)$

02 Given a no. , we need to print all the prime no. from 1 to N.

$N = 10 \rightarrow 2, 3, 5, 7$

$N = 20 \rightarrow 2, 3, 5, 7, 11, 13, 17, 19$

Idea $\rightarrow$ Iterate from 1 to N &

check if it is prime or not.

```
void printall (int N)
```

```
    for (i=1; i ≤ N; i++) {
```

```
        if (checkprime(i) == True) {
```

```
            print(i);
```

Tc: $O(N\sqrt{N})$

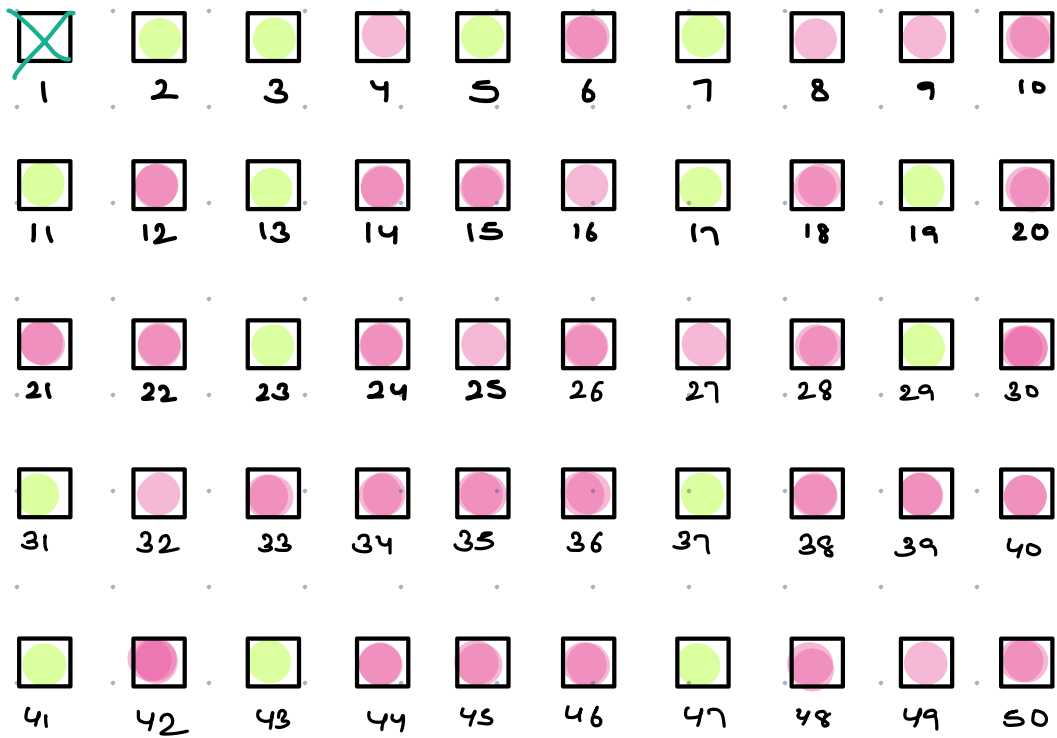
Sc: $O(1)$

Idea 2

Sieve of eratosthenes

Given $n = 50$, we have to find all primes between 1 to 50.

Assumption $\rightarrow$ Every no. is a prime



If 2 is a prime, what all no. can never be the prime no.

→ Multiples of 2 will never be a prime no.

```
void printallprimes (int n)
```

```
    boolean [] prime = new int [n+1];
```

```
    // mark every ele as True;
```

```
    Arrays.fill (prime, True);
```

```
    prime [0] = false
```

```
    prime [1] = false
```

```
for (i = 2; i ≤ n; i++) {
```

```
    if (prime[i] == true) {
```

```
        for (j = i * i; j ≤ n; j = j + i) {
```

```
            prime[j] = false;
```

3

3

3

```
for (i = 2; i ≤ N; i++) {
```

```
    if (prime[i] == True) print(i);
```

3

3

i

Multiples of i & Mark them as false

2

2 * 2

2 * 3

2 * 4

2 * 5

2 * 6 ...

3

3 * 2

3 * 3

3 * 4

3 * 5

3 * 6 ...

4

5

5 * 2

5 * 3

5 * 4

5 * 5

5 * 6 ...

Obs $\rightarrow$ Instead of going on multiples from $2i$,
we will iterate from $i \neq i$

$i [2 \rightarrow n]$

$j [i \neq i \rightarrow n]$

2

$\approx$

$n/2$ iterations

3

$\approx$

$n/3$ iterations

4

5

$\approx$

$n/5$ iterations

.

.

.

$\sqrt{n}$

$j : [(\sqrt{n})^2 \rightarrow n] = 1$ iteration

$\sqrt{n} + 1$

$j : [(\sqrt{n} + 1)^2 \rightarrow n] = 0$ iteration

$\sqrt{n} + 2$

0 iteration

.

.

n

0 iterations

$$\pi_c = \frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots$$

$$= 1 \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots \right)$$

Sum of all reciprocals of prime no.

$$= N * (\log(\log N))$$

$$Tc: O(N \log(\log N))$$

$$Sc: O(N)$$

* Given N , return the smallest prime factor for all no. from 2 to N .

$N = 10$ 2 3 4 5 6 7 8 9 10

Ans

2 3 2 5 2 7 2 3 2

Ass $\rightarrow$ Every no. is spf of itself

1	2	3	2	5	2	7	2	3	2
1	2	3	4	5	6	7	8	9	10
11	2	13	2	3	2	17	2	19	2
11	12	13	14	15	16	17	18	19	20
3	2	23	2	5	2	3	2	29	2
21	22	23	24	25	26	27	28	29	30
31	2	3	2	5	2	37	2	3	2
31	32	33	34	35	36	37	38	39	40
41	2	43	2	3	2	47	2	7	2
41	42	43	44	45	46	47	48	49	50

Observations

01. Identify prime $\rightarrow$ if $(A[i] == i) \rightarrow$ prime no

02. we should not update ele which have been already updated $\rightarrow$ if $(A[i] != i) \rightarrow$ already updated.

int [] getspf (int n)

int [] spf = new int [n+1];

for all i , $spf[i] = i$

for ($i = 2$; $i \leq n$; $i++$) {

↓

spf[i] = i;

Tc: $O(n \log(\log n))$

Sc: $O(1)$

for ($i = 2$; $i * i \leq n$; $i++$) {

if (spf[i] == i) { // checking if i is prime

for ($j = i * i$; $j \leq n$; $j = j + i$) {

if (spf[j] == j) {

↓

spf[j] = i;

3

3

3

return spf;

8:13 AM $\rightarrow$ 8:23 AM

* prime factorisation

→ process of finding prime no., which are multiplied together to form the given no.

$$n = 48$$

2	48
2	24
2	12
2	6
3	3
	1

$$n = 48 = 2 \times 2 \times 2 \times 2 \times 3$$

$$= 2^4 \times 3^1$$

$$\text{No. of factors} = (4+1) \times (1+1) = \underline{\underline{10}}$$

Factors of 48 = 1, 2, 3, 4, 6, 8, 12, 16, 24, 48

$$\underline{\underline{n = 300}}$$

2	300
2	150
3	75
5	25
5	5
	1

$$n = 300$$

$$2 \times 2 \times 3 \times 5 \times 5$$

$$= 2^2 \times 3^1 \times 5^2$$

$$\text{No. of factors} = (2+1) \times (1+1) \times (2+1)$$

$$= 3 \times 2 \times 3$$

$$= \underline{\underline{18}}$$

* Given a no. & its prime factors

$$n = i^{a_1} * j^{a_2} * k^{a_3}$$

$$\text{No. of factors} = (a_1 + 1) * (a_2 + 1) * (a_3 + 1)$$

* Given a no. n , for all the no. from 1 to N , get the no. of factors.

$N=10$	1	2	3	4	5	6	7	8	9	10
	1	1	1	1	1	1	1	1	1	1
		2	3	2	5	2	1	2	3	2
				4		3		4	9	5
						6		8		10

Ans

1 2 2 3 2 4 2 4 3 4

$N=360$

2	360
2	180
2	90
3	45
3	15

$$\left. \begin{aligned} \text{spf}[360] &= 2 \\ \text{spf}[180] &= 2 \\ \text{spf}[90] &= 2 \end{aligned} \right\}$$

$$\text{spf}[45] = 3$$

5	5
	1

$$\text{spf}[15] = 3$$

$$\text{spf}[5] = 5$$

Steps

01 get spf[]

$$2^3 * 3^2 * 5^1$$

$$\text{Ans} = (3+1) * (2+1) * (1+1)$$

$$= 4 * 3 * 2$$

$$= 24$$

```
int [] spf = getSpf[N]
```

```
ans = 1
```

```
while (N > 1) {
```

```
    s = spf[N]
```

```
    count = 0
```

```
    while (N % s == 0) {
```

```
        N = N / s
```

```
        count ++;
```

```
    }
    ans = ans * (count + 1)
```

No. of factors
for one value of
N

TC: $O(N \log(\log N)) + O(\log n)$
SC: $O(n)$

* H.W $\Rightarrow$ You have to run this process for multiple inputs.

Doubts

Whenever you are give with mod

= making some changes

$ans = (ans + k) \% mod$ X

$ans = (ans \% mod + k \% mod) \% mod$

```
public class Solution {
    public int solve(int A, int B, int C) {
        // dp[n][r] stores the value of nCr
        int[][] dp = new int[A + 1][B + 1];
        for(int i = 0; i <= A; i++) {
            for(int j = 0; j <= Math.min(i, B); j++) {
                if(j == i || j == 0) {
                    dp[i][j] = 1;
                } else {
                    // nCr = (n - 1)C(r - 1) + (n - 1)Cr
                    dp[i][j] = (dp[i - 1][j - 1] % C + dp[i - 1][j] %
C) % C;
                }
            }
        }
        return dp[A][B] % C;
    }
}
```